NANYANG TECHNOLOGICAL UNIVERSITY SINGAPORE

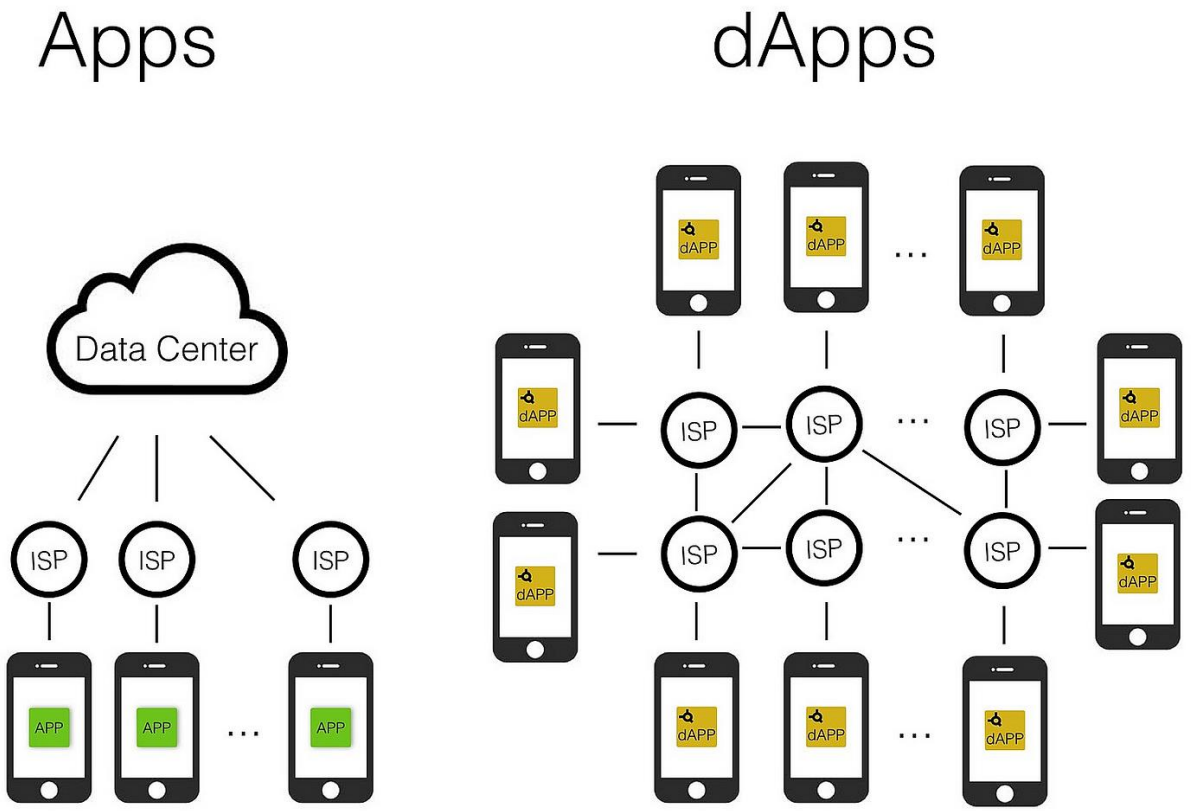
CE/CZ4153 Tutorial

dApp Development Workshop

Zhang Chengxuan
chengxua001@e.ntu.edu.sg
2025.9.9

What is dApp?

- Decentralized applications powered by blockchain.



Source: https://towardsdatascience.com/what-is-a-dapp-a455ac5f7def

Why developing dApp? Reliable Service

- Traditional Network Applications

  C/S Architecture, e.g., Zoom desktop App.

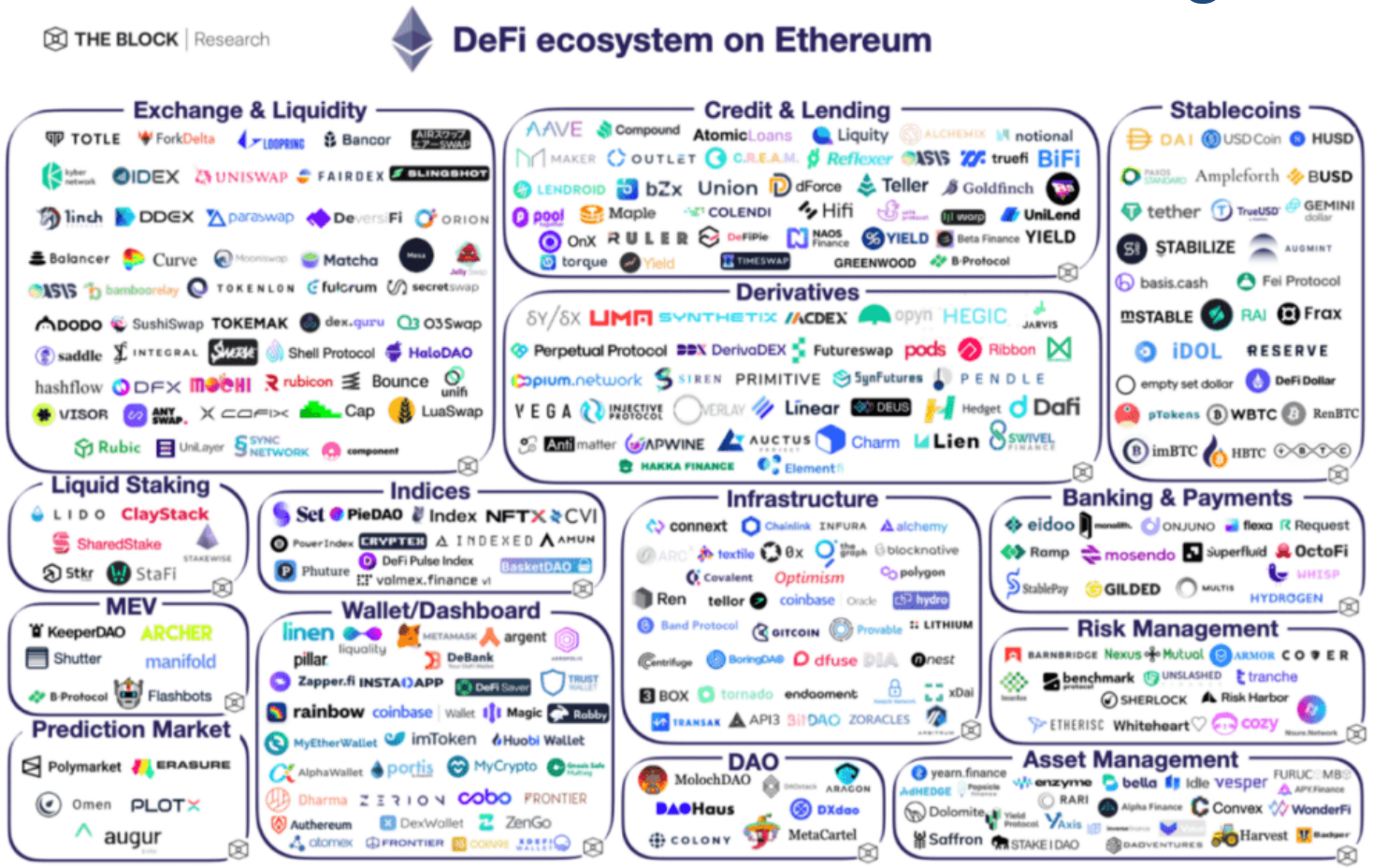
  B/S Architecture, e.g., www.ntu.edu.sg

  Issue: distributed/centralized server may crash sometimes

- Blockchains like Bitcoin or Ethereum have seamlessly worked for many years without any crash issue.

Why developing dApp? New Opportunities!

- Blockchain is a network for value exchange among users.



Source: https://www.btcfans.com/en-us/article/72488

How to develop a dApp?

- **dApp** = Smart contracts + Javascript/HTML
- **Backend** - Smart contracts implement business logic and are run on one or many blockchains.
- **Frontend** - Javascript / HTML implements the UI of the dApp and communicates with smart contracts through sending blockchain transactions.

Three transaction paths

LOCK

React + MetaMask

User sees and approves each write

ELECTION

React + Express

Server chooses and signs

TOKENIZATION

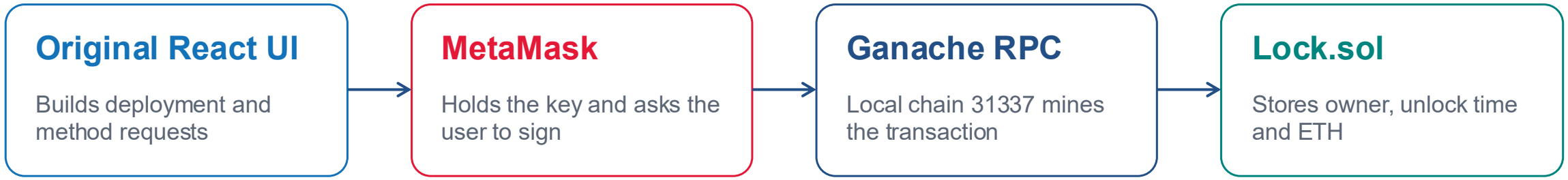
Next.js + wallet

Burner wallet or MetaMask signs

Lock dApp

- Lock a certain amount of money with a given timestamp.
- In the future, users can withdraw their money back.
- Moreover, the owner can unlock the money if they want.

Lock frontend, wallet and contract



Reads show state without mining. Deploy, unlock and withdraw each create a wallet confirmation and a receipt.

Lock dApp

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.9;

// Import this file to use console.log
import "hardhat/console.sol";

contract Lock {
    uint public unlockTime;
    address payable public owner;

    event Withdrawal(uint amount, uint when);

    constructor(uint _unlockTime) payable {
        require(
            block.timestamp < _unlockTime,
            "Unlock time should be in the future"
        );

        unlockTime = _unlockTime;
        owner = payable(msg.sender);
    }
    function unlock() public{
        require(msg.sender == owner, "You aren't the owner");
        unlockTime = block.timestamp;
    }

    function withdraw() public {
        // Uncomment this line to print a log in your terminal
        // console.log("Unlock time is %o and block timestamp is %o", unlockTime, block.timestamp);

        require(block.timestamp >= unlockTime, "You can't withdraw yet");
        require(msg.sender == owner, "You aren't the owner");

        emit Withdrawal(address(this).balance, block.timestamp);

        owner.transfer(address(this).balance);
    }
}
```

Lock dApp

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.9;
```

Solidity Version Later than 0.8.9

```
// Import this file to use console.log
import "hardhat/console.sol";
```

Import External Library

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.9;

// Import this file to use console.log
import "hardhat/console.sol";

contract Lock {
    uint public unlockTime;
    address payable public owner;

    event Withdrawal(uint amount, uint when);

    constructor(uint _unlockTime) payable {
        require(
            block.timestamp < _unlockTime,
            "Unlock time should be in the future"
        );

        unlockTime = _unlockTime;
        owner = payable(msg.sender);
    }
    function unlock() public{
        require(msg.sender == owner, "You aren't the owner");
        unlockTime = block.timestamp;
    }

    function withdraw() public {
        // Uncomment this line to print a log in your terminal
        // console.log("Unlock time is %o and block timestamp is %o", unlockTime, block.timestamp);

        require(block.timestamp >= unlockTime, "You can't withdraw yet");
        require(msg.sender == owner, "You aren't the owner");

        emit Withdrawal(address(this).balance, block.timestamp);

        owner.transfer(address(this).balance);
    }
}
```

Lock dApp

```
7     contract Lock {
8         uint public unlockTime;
9         address payable public owner;
10
11        event Withdrawal(uint amount, uint when);
12
13        constructor(uint _unlockTime) payable {
14            require(
15                block.timestamp < _unlockTime,
16                "Unlock time should be in the future"
17            );
18
19            unlockTime = _unlockTime;
20            owner = payable(msg.sender);
21        }
```

Contract Name: Lock

Variable Declarations

Event Declaration

Condition Checking

```
1   // SPDX-License-Identifier: UNLICENSED
2   pragma solidity ^0.8.9;
3
4   // Import this file to use console.log
5   import "hardhat/console.sol";
6
7   contract Lock {
8       uint public unlockTime;
9       address payable public owner;
10
11      event Withdrawal(uint amount, uint when);
12
13      constructor(uint _unlockTime) payable {
14          require(
15              block.timestamp < _unlockTime,
16              "Unlock time should be in the future"
17          );
18
19          unlockTime = _unlockTime;
20          owner = payable(msg.sender);
21      }
22      function unlock() public{
23          require(msg.sender == owner, "You aren't the owner");
24          unlockTime = block.timestamp;
25      }
26
27      function withdraw() public {
28          // Uncomment this line to print a log in your terminal
29          // console.log("Unlock time is %o and block timestamp is %o", unlockTime, block.timestamp);
30
31          require(block.timestamp >= unlockTime, "You can't withdraw yet");
32          require(msg.sender == owner, "You aren't the owner");
33
34          emit Withdrawal(address(this).balance, block.timestamp);
35
36          owner.transfer(address(this).balance);
37      }
38  }
```

Lock dApp

```
function unlock() public{
    require(msg.sender == owner, "You aren't the owner");
    unlockTime = block.timestamp;
}

function withdraw() public {
    // Uncomment this line to print a log in your terminal
    // console.log("Unlock time is %o and block timestamp is %o", unl

    require(block.timestamp >= unlockTime, "You can't withdraw yet");
    require(msg.sender == owner, "You aren't the owner");

    emit Withdrawal(address(this).balance, block.timestamp);

    owner.transfer(address(this).balance);
}
```

Condition Checking

Condition Checking

Emit Event

Transfer Token Balance

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.9;

// Import this file to use console.log
import "hardhat/console.sol";

contract Lock {
    uint public unlockTime;
    address payable public owner;

    event Withdrawal(uint amount, uint when);

    constructor(uint _unlockTime) payable {
        require(
            block.timestamp < _unlockTime,
            "Unlock time should be in the future"
        );

        unlockTime = _unlockTime;
        owner = payable(msg.sender);
    }
    function unlock() public{
        require(msg.sender == owner, "You aren't the owner");
        unlockTime = block.timestamp;
    }

    function withdraw() public {
        // Uncomment this line to print a log in your terminal
        // console.log("Unlock time is %o and block timestamp is %o", unlockTime, block.timestamp);

        require(block.timestamp >= unlockTime, "You can't withdraw yet");
        require(msg.sender == owner, "You aren't the owner");

        emit Withdrawal(address(this).balance, block.timestamp);

        owner.transfer(address(this).balance);
    }
}
```

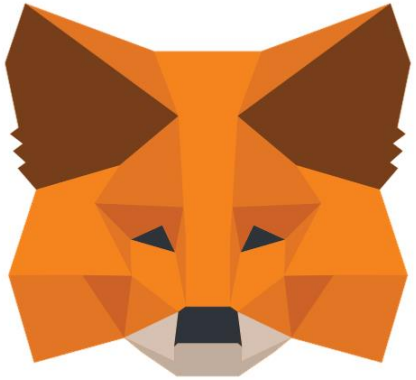
Lock live demo with MetaMask

1 **Connect to Localhost 8545, chain ID 31337**

2 Deploy with a future UTC time and small wei value

3 Confirm owner unlock and withdrawal

4 Verify the contract balance becomes zero

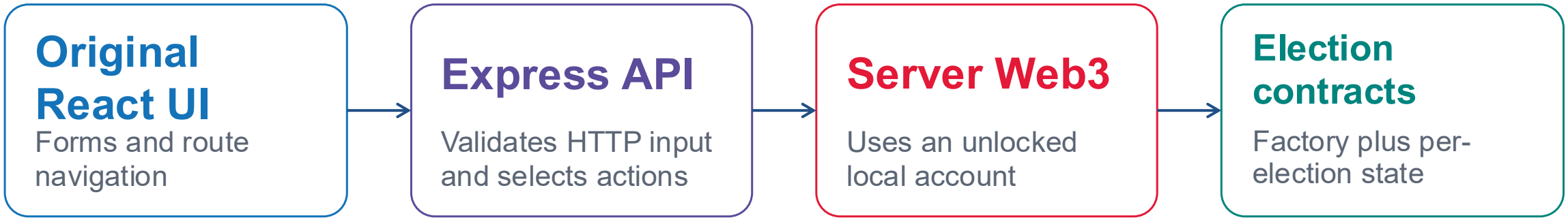


Every write has two visible stages: wallet approval and mined confirmation.

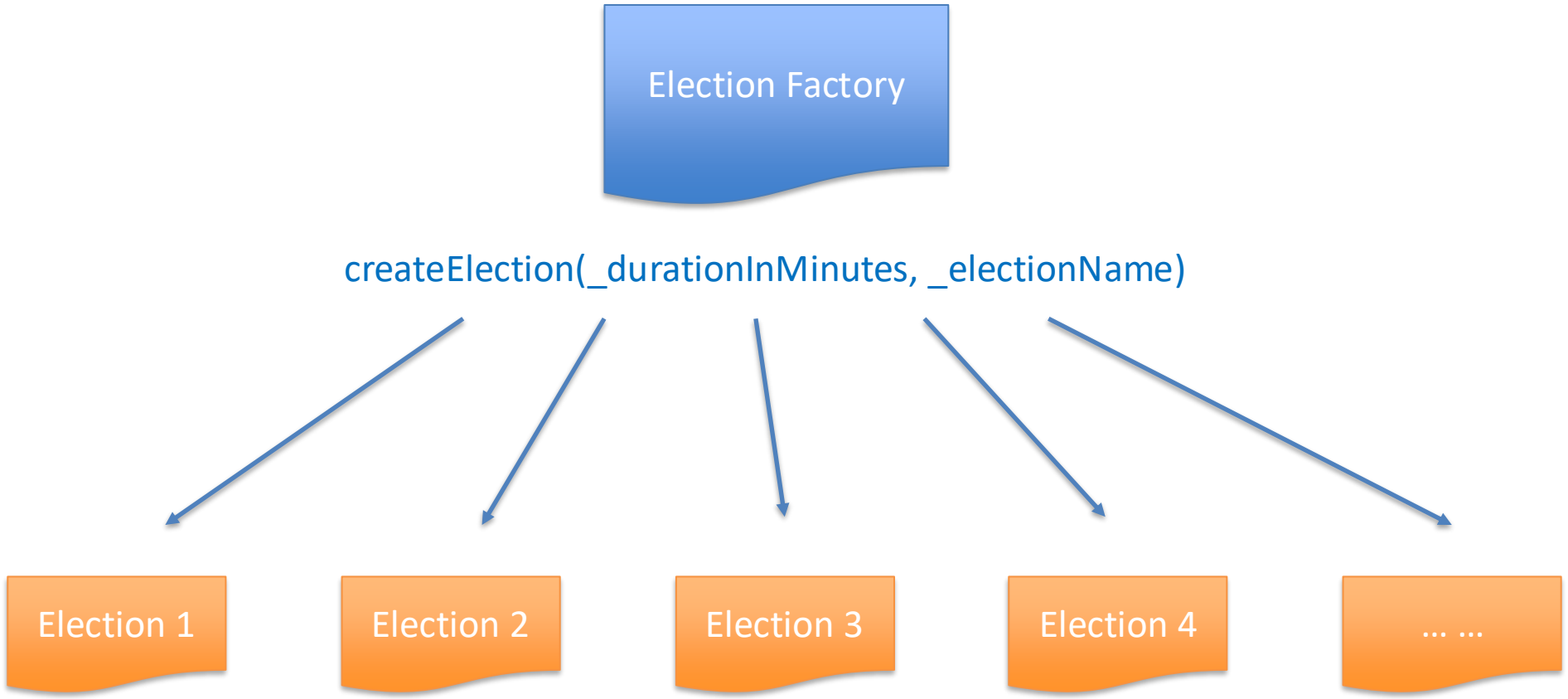
Election dApp

- Allows creation of multiple elections with a defined duration and admin control.
- Admin registers constituencies, voters, and candidates, ensuring no self-registration.
- Voters cast a single vote for candidates within their constituency.
- Admin closes the election after the set duration, ending voting.
- Tracks and displays votes for each candidate by constituency.

Election frontend and backend



Election dApp



Election

Variables

admin
electionName
electionStatus
electionDuration
votersList
voterExist
voterData
candidateList
candidateExist
candidateData
consituencyList
consituencyExist
consituencyData

Structs

Voter
- voterId
- name
- email
- phoneNo
- consituencyId
- age
- voted

Candidate
- candidateId
- name
- email
- phoneNo
- consituencyId
- party

Consituency
- consituencyId
- name
- candidates
- voters
- votes (mapping of address to uint)
- winner

Functions

Add
- addConsituency
- addCandidate
- addVoter

Get
- getCandidatesIdList
- getConsituencyCandidates
- getCandidateConsituency
- getConsituencyIdList
- getVotersIdList
- getConsituencyVoters
- getVoterConsituency

Vote
- castVote
- getVotes

Close
- closeElection

https://github.com/schadokar/election-ethereum-react-dapp/blob/master/ethereum/contracts/election.sol

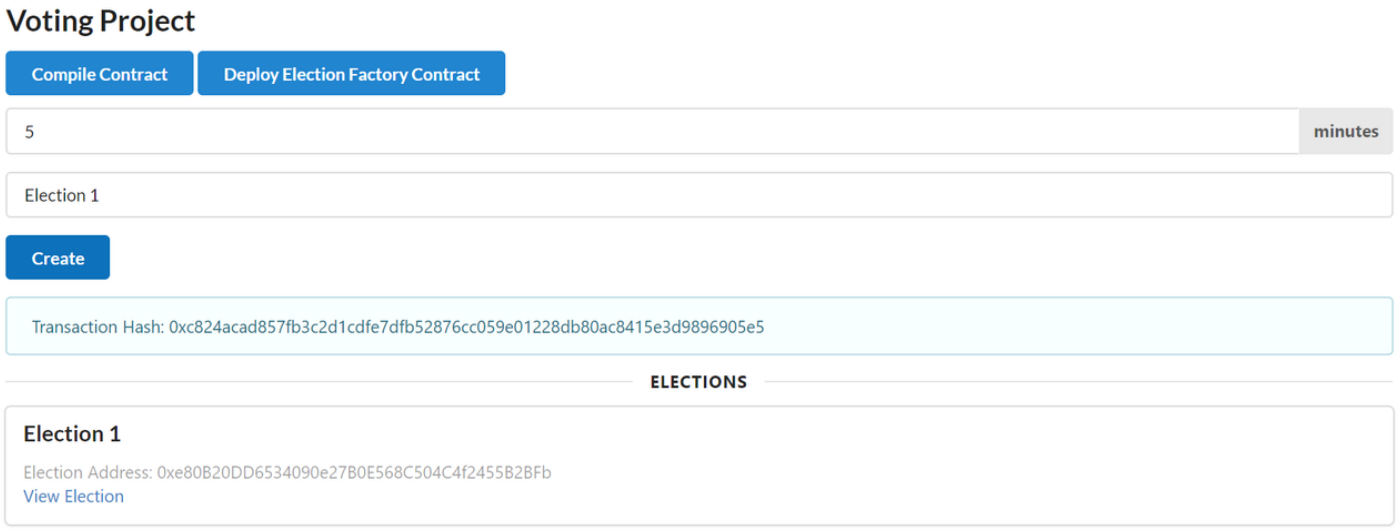
Election original UI demo

Create an election

Open its contract address

Add a constituency

Trace React → Express → Web3 → contract



ERC721 Scaffold-ETH 2 Tokenization

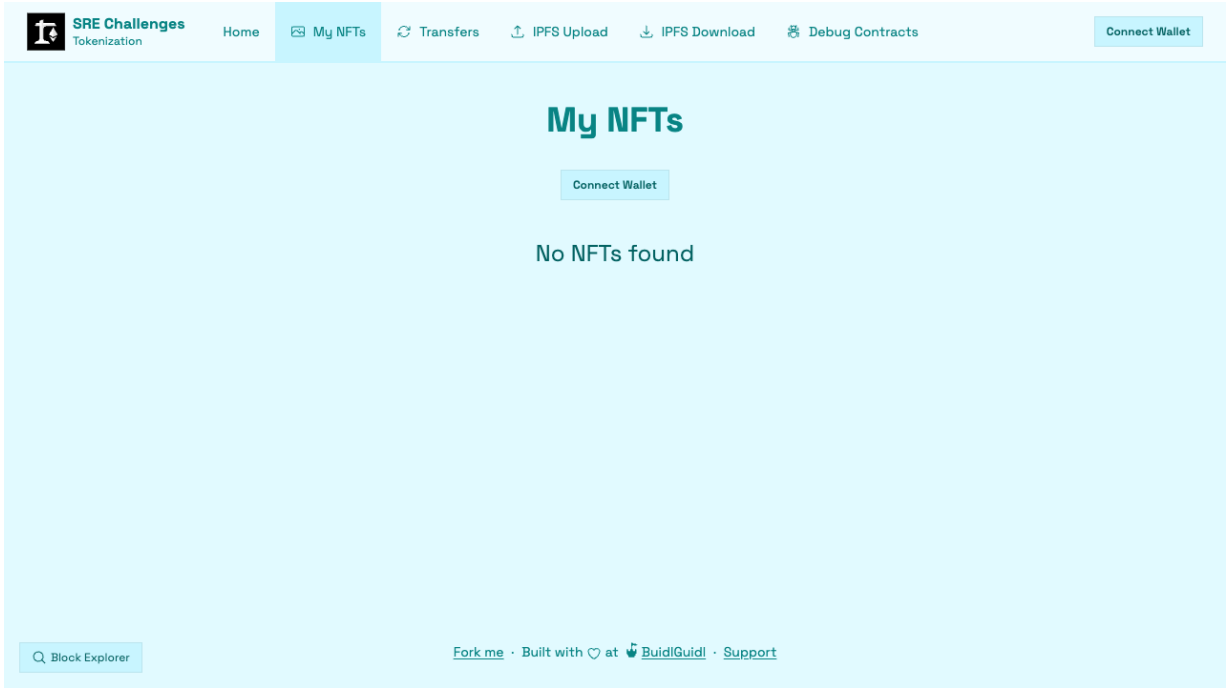
Official teaching project

Next.js + RainbowKit + wagmi

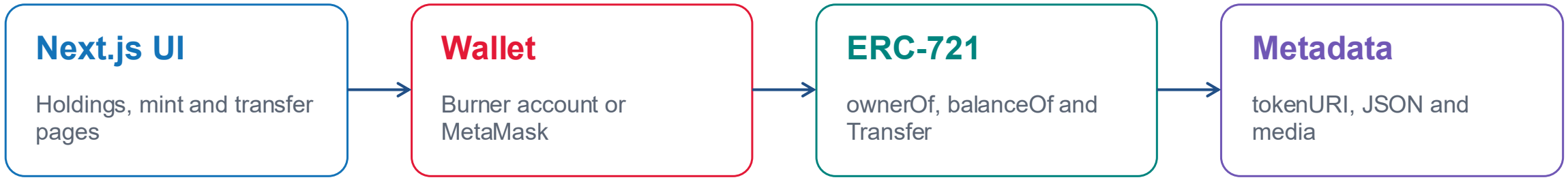
Hardhat + OpenZeppelin ERC-721

Minting, transfers, events and IPFS exercises

Maintained tests and Debug Contracts



Tokenization frontend and onchain evidence



Ownership is verified onchain. The description and image can live elsewhere.

For a real-world asset, define who issues the token and what legal right it represents.

Mint, transfer and explain the evidence

1 **MINT**

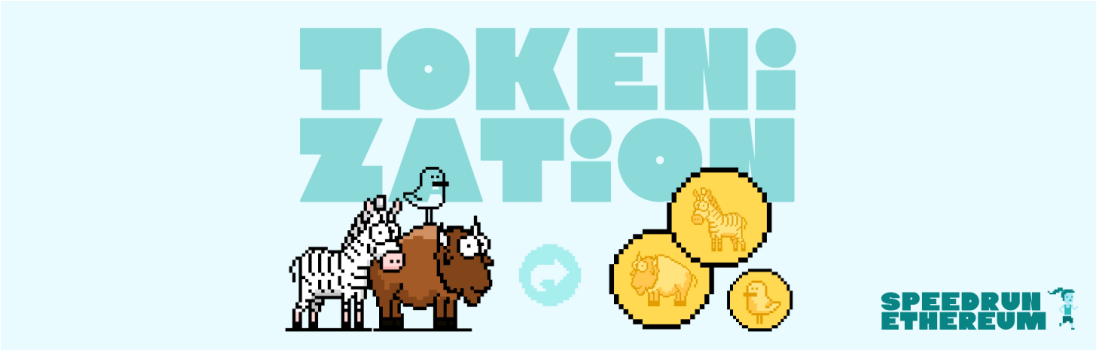
A transaction creates token #n

2 **TRANSFER**

Ownership moves to the second address

3 **VERIFY**

ownerOf and Transfer event agree



The screen, contract read and event history explain the same ownership change.

One-command local stack

./run-lock.command	Ganache + original React	**MetaMask signs**
./run-election.command	Ganache + Express + original React	**Server Web3 signs**
./run-tokenization.command	Hardhat + official Next.js	**Burner wallet or MetaMask signs**

Ctrl+C stops the complete stack and the next run creates a fresh chain.

Architecture comparison and questions

LOCK

User-controlled key MetaMask confirmation

ELECTION

Trusted server key No browser approval

TOKENIZATION

Burner or MetaMask Ownership plus metadata

https://github.com/cxzhang93/dapp-workshop